

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая кибернетика и информационные технологии»

Отчет по лабораторной работе №4

по дисциплине «Информационные технологии и программирование»

Выполнил: студент группы

БПИ2401

Старков Дмитрий Константинович

Проверил:

Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы	2
Ход работы.....	2
Вывод.....	5

Цель работы

Освоить исключения в языке Java.

Ход работы

Начнем с выполнения первого задания:



```
1 package lab4;
2
3 public class Exceptions_Array {
4     public static void main(String[] args) {
5         int[] array = {1, 2, 3, 4, 5};
6         int sum = 0;
7         try {
8             for (int i = 0; i <= array.length; i++) {
9                 sum += array[i];
10            }
11        } catch (ArrayIndexOutOfBoundsException e) {
12            System.out.println("Error: Attempted to access an index outside the array bounds.");
13        } catch (Exception e) {
14            System.out.println("An unexpected error occurred: " + e.getMessage());
15        } finally {
16            System.out.println("Final sum is: " + sum);
17        }
18    }
19 }
20
```

После инициализации массива и суммы я сразу пишу блок try-catch, и в блоке try намеренно делаю так, чтобы цикл выходил за длину массива и тем самым вызывал ошибку. Поэтому в блоке catch я указываю как раз ошибку `ArrayIndexOutOfBoundsException` и вывожу на экран сообщение об этой ошибке. Во втором блоке catch я обрабатываю прочие исключения, поэтому в сообщении об ошибке есть `e.getMessage()` чтобы вывести конкретную ошибку.

И в конце блок finally, который выводит посчитанную сумму.

Теперь второе задание

```
1 package lab4;
2
3 public class Exceptions_Files {
4     public static void main(String[] args) {
5         try {
6             java.io.FileInputStream in = new java.io.FileInputStream("file1.txt");
7             java.io.FileOutputStream out = new java.io.FileOutputStream("file2.txt");
8
9             try {
10                byte[] buffer = new byte[1024];
11                int length;
12
13                while ((length = in.read(buffer)) > 0) {
14                    out.write(buffer, 0, length);
15                }
16
17                System.out.println("File copied successfully!");
18
19                } catch (java.io.IOException e) {
20                    System.err.println("Error while reading/writing file: " + e.getMessage());
21                } finally {
22                    try {
23                        in.close();
24                        out.close();
25                    } catch (java.io.IOException e) {
26                        System.err.println("Error while closing files: " + e.getMessage());
27                    }
28                }
29
30            } catch (java.io.FileNotFoundException e) {
31                System.err.println("Error opening file: " + e.getMessage());
32            }
33        }
34    }
```

Здесь я обрабатывал возможные ошибки при копировании содержания одного файла в другой. Именно поэтому у меня есть вложенный блок try-catch. Во внешнем блоке я инициализирую файлы для считывания и записи, соответственно на этом уровне единственная возможная ошибка — это невозможность найти файл в файловой системе. Во внутреннем блоке я инициализирую буфер на 1кб, чтобы считывать файл кусками, а также длину. И в цикле while я читаю первый килобайт информации в файле, после чего записываю его в файл для записи, начиная с 0 элемента буфера и записывая ровно length символов из него.

На этом этапе может возникнуть IOException(Input-Output) если файлы были повреждены, изменены или подвергнуты прочему влиянию во время процесса чтения/записи. Такая же ошибка могла возникнуть и при закрытии файлов, поэтому у меня 2 вложенных блока try-catch.

В третьем задании необходимо было реализовать собственное исключение.

```
1  package lab4;
2
3  public class CustomAgeException {
4      public static void main(String[] args) {
5          String age_raw = "0";
6
7          try {
8              int age = Integer.parseInt(age_raw);
9              if (age <= 0 || age > 120 || age != (int) age) {
10                 throw new AgeException("type a valid age");
11             } else {
12                 System.out.println("Age is valid.");
13             }
14         } catch (AgeException e) {
15             Logger.log(e);
16             System.err.println("Custom Exception Caught: " + e.getMessage());
17         } catch (NumberFormatException e) {
18             Logger.log(e);
19             System.err.println("Invalid input format: " + e.getMessage());
20         }
21     }
22 }
23
24 class AgeException extends Exception {
25     public AgeException(String message) {
26         super(message);
27     }
28 }
29
30 class Logger {
31
32     private static final String LOG_FILE = "errors.log";
33
34     public static void log(Exception e) {
35         try (java.io.FileWriter fw = new java.io.FileWriter(LOG_FILE, true)) {
36             fw.write(e.toString() + "\n");
37         } catch (java.io.IOException ioEx) {
38             System.err.println("Log error: " + ioEx.getMessage());
39         }
40     }
41 }
```

Помимо собственного исключения, нужно было выводить информацию об ошибках в специальный файл с логами. Именно поэтому я начну с класса `Logger`, который за это отвечает. В нем нет ничего сложного – просто записать сообщения об ошибке с новой строки, либо вывод ошибки в консоль при неудачной попытке записи ошибки в логи. А еще уточню, что `true` во втором аргументе `FileWriter` нужен для того, чтобы ошибки записывались в конец файла, а не перезаписывали его.

Далее, если идти снизу вверх, идет класс AgeException, который наследует класс Exceptions, именно поэтому мой класс состоит из одного конструктора, который обращается к конструктору родительского класса.

И наконец сам код программы

В нем я учитываю 2 возможные ошибки: первое – возраст не попадает в рамки от 0 до 120 лет и не является целым числом (для этой ошибки я создал собственное исключение), и второе – возраст не является числом вовсе (для этого я воспользовался уже существующим исключением

NumberFormatException и вызвал уже его. Соответственно у меня в коде два блока catch для каждой из этих ситуаций.

Вывод

В ходе лабораторной работы мной были освоены исключения в языке Java.

Ссылка на гит - <https://github.com/BestStarProgrammer/IT-P/tree/main/lab2>